TL09 Aumento de datos

Índice

1 Introducción

Aumento de datos: técnica de regularización convencional; con datos reales o sintéticos (reales perturbados)

Aplicación a imágenes: post-entrenamiento con imágenes perturbadas mediante transformaciones razonables

Modelo pre-entrenado: usaremos un modelo `timm` pre-entrenado

- Conviene conocer el dataset con el que se ha pre-entrenado; típicamente ImageNet
- Debemos considerar la transformación específica del modelo pre-entrenado

Dataset de post-entrenamiento: usaremos CIFAR-10, MNIST, Fashion-MNIST, etc.

- Conviene conocerlo y compararlo con el de pre-entrenamiento
- Necesitamos conocer el formato original de las imágenes
- Se procesa en batches; en streaming si no se quiere descargar completo

Perturbación de imágenes de entrenamiento: usaremos torchvision.transforms.v2 de torchvision

1. `ToImage()` y `ToDtype` a `torch.uint8` para convertir imágenes en tensores
2. `Resize`, `CenterCrop`, `RandomHorizontalFlip`, etc.
3. `ToDtype` a `torch.float32` y `Normalize` para estandarizar imágenes

Imágenes de test: no deben perturbarse; basta convertirlas a tensores `torch.float32` y estandarizar

Objetivo: familiarizarse con técnicas de aumento de datos populares

2 transforms v2

Imagen ejemplo: en formato JPEG

In [ ]:
```
import matplotlib.pyplot as plt; import urllib; from PIL import Image
url = "https://github.com/pytorch/hub/raw/master/images/dog.jpg"; file = "dog.jpg"
urllib.request.urlretrieve(url, file); img = Image.open(file).convert("RGB"); img.reduce(4)
```

Out[ ]:

Composición de transformaciones: Compose aplica una a una las transformaciones de una lista dada

Conversión a `tensor.uint8`: ToImage() y ToDtype(dtype, scale=True)

- `ToImage` convierte un tensor, ndarray, o imagen PIL en Image
- `ToDtype` convierte la entrada a un dtype específico y, opcionalmente, escala valores; p.e. a `[0,1]` para `float`

In [ ]:
```
import torch; from torchvision.transforms import v2
transforms = v2.Compose([v2.ToImage(), v2.ToDtype(torch.uint8, scale=True)])
out = transforms(img); plt.xticks([]); plt.yticks([]); plt.imshow(out.permute(1, 2, 0).numpy());
```

Reflejo horizontal aleatorio: RandomHorizontalFlip(p=0.5)

- `p` es la probabilidad de reflejar horizontalmente

In [ ]:
```
transforms = v2.Compose([v2.ToImage(), v2.ToDtype(torch.uint8, scale=True), v2.RandomHorizontalFlip(p=1.0)])
out = transforms(img); plt.xticks([]); plt.yticks([]); plt.imshow(out.permute(1, 2, 0).numpy());
```

Redimensión: Resize(size, interpolation=InterpolationMode.BILINEAR, max_size=None, antialias=True)

- `size` puede ser `(H, W)`, `(int,)` (lado menor) o `None` (se emplea `max_size`)
- `antialias=True` antialising con interpolación bilineal o bicúbica
- `max_size=None` es la dimensión máxima del lado más largo de la imagen redimensionada

In [ ]:
```
transforms = v2.Compose([v2.ToImage(), v2.ToDtype(torch.uint8, scale=True), v2.RandomHorizontalFlip(p=1.0),
    v2.Resize(size=(36,), max_size=None)])
out = transforms(img); plt.xticks([]); plt.yticks([]); plt.imshow(out.permute(1, 2, 0).numpy());
```

Recorte central: CenterCrop(size)

- `size` puede ser `int` (cuadrada) o una secuencia, p. ej. `(H, W)`

In [ ]:
```
transforms = v2.Compose([v2.ToImage(), v2.ToDtype(torch.uint8, scale=True), v2.RandomHorizontalFlip(p=1.0),
    v2.Resize(size=36, max_size=None), v2.CenterCrop(size=32)])
out = transforms(img); plt.xticks([]); plt.yticks([]); plt.imshow(out.permute(1, 2, 0).numpy());
```

Normalización: Normalize(mean, std, inplace=False)

- `mean` es una secuencia de medias por canal
- `std` es una secuencia de desviaciones típicas por canal
- `inplace=False` indica si se quiere hacer in-place o no

In [ ]:
```
transforms = v2.Compose([v2.ToImage(), v2.ToDtype(torch.uint8, scale=True), v2.RandomHorizontalFlip(p=1.0),
    v2.Resize(size=36, max_size=None), v2.CenterCrop(size=(32, 32)), v2.ToDtype(torch.float32, scale=True),
    v2.Normalize(mean=(.0, .0, .0), std=(1., 1., 1.))])
out = transforms(img); plt.xticks([]); plt.yticks([]); plt.imshow(out.permute(1, 2, 0).numpy());
```

3 Ejemplo

`exp.py` : rutina experimental para schedulers no guiados por una métrica

In [ ]:
```python
import torch

def exp(model, device, train_loader, test_loader, optimizer, scheduler, epochs=15):
    loss_fn = torch.nn.CrossEntropyLoss()
    for epoch in range(epochs):
        print(f'Epoch {epoch}: train:', end=' ')
        model.train(); trsize, trbatches, trloss, tracc = 0, 0, 0, 0
        for batch in train_loader:
            X, y = batch['X'].to(device), batch['y'].to(device); trsize += len(X); trbatches += 1
            pred = model(X); loss = loss_fn(pred, y); trloss += loss.item()
            tracc += (pred.argmax(1) == y).type(torch.float).sum().item()
            loss.backward(); optimizer.step(); optimizer.zero_grad(); scheduler.step()
        trloss /= trbatches; tracc /= trsize
        print(f'loss {trloss:g} acc {tracc:.2%} test:', end=' ')
        model.eval(); tesize, tebatches, teloss, teacc = 0, 0, 0, 0
        with torch.no_grad():
            for batch in test_loader:
                X, y = batch['X'].to(device), batch['y'].to(device); tesize += len(X); tebatches += 1
                pred = model(X); teloss += loss_fn(pred, y).item()
                teacc += (pred.argmax(1) == y).type(torch.float).sum().item()
        teloss /= tebatches; teacc /= tesize
        print(f'loss {teloss:g} acc {teacc:.2%}')
```

Ejemplo:

- CIFAR10 en HF: `uoft-cs/cifar10`

In [ ]:
```
import datasets; from torch.utils.data import DataLoader
ds = datasets.load_dataset("uoft-cs/cifar10").rename_columns({'img': 'X', 'label': 'y'})
train_ds = ds['train'].to_iterable_dataset(num_shards=1024).shuffle()
test_ds = ds['test'].to_iterable_dataset(num_shards=1024)
```

- Modelo `timm` : `resnet50.fb_swsl_ig1b_ft_in1k`

In [ ]:
```
import numpy as np; import matplotlib.pyplot as plt; import torch; import timm; from exp import exp
device = torch.accelerator.current_accelerator().type if torch.accelerator.is_available() else "cpu"; device
model_name = "resnet50.fb_swsl_ig1b_ft_in1k" # avg_top1 71.5 param_count 25.56
model = timm.create_model(model_name, pretrained=True, num_classes=10).to(device)
```

- Transformación específica:

In [ ]:
```
data_cfg = timm.data.resolve_data_config(model.pretrained_cfg); data_cfg
```

Out[ ]:
```
{'input_size': (3, 224, 224),
 'interpolation': 'bilinear',
 'mean': (0.485, 0.456, 0.406),
 'std': (0.229, 0.224, 0.225),
 'crop_pct': 0.875,
 'crop_mode': 'center'}
```

- Transformación específica adaptada al tamaño de las imágenes de CIFAR10:

In [ ]:
```
data_cfg['input_size'] = (3, 32, 32)
timm.data.create_transform(**data_cfg)
```

Out[ ]:
```
Compose(
    Resize(size=36, interpolation=bilinear, max_size=None, antialias=True)
    CenterCrop(size=(32, 32))
    MaybeToTensor()
    Normalize(mean=tensor([0.4850, 0.4560, 0.4060]), std=tensor([0.2290, 0.2240, 0.2250]))
)
```

- Transformación específica para entrenamiento y test:

In [ ]:
```
from torchvision.transforms import v2
train_transform = v2.Compose([
    v2.ToImage(),
    v2.ToDtype(torch.uint8, scale=True),
    v2.RandomHorizontalFlip(),
    # v2.Resize(size=36, max_size=None),
    # v2.CenterCrop(size=(32, 32)),
    v2.ToDtype(torch.float32, scale=True),
    v2.Normalize(mean=data_cfg['mean'], std=data_cfg['std'])])
def train_prep(example):
    return { 'X' : train_transform(example['X']), 'y' : example['y']}
train_ds = train_ds.map(train_prep, batched=True)
train_loader = DataLoader(train_ds, batch_size=32)
```

In [ ]:
```
test_transform = v2.Compose([
    v2.ToImage(),
    v2.ToDtype(torch.float32, scale=True),
    v2.Normalize(mean=data_cfg['mean'], std=data_cfg['std'])])
def test_prep(example):
    return { 'X' : test_transform(example['X']), 'y' : example['y']}
test_ds = test_ds.map(test_prep, batched=True)
test_loader = DataLoader(test_ds, batch_size=32)
```

- Experimento:

In [ ]:
```
optimizer = torch.optim.AdamW(filter(lambda p: p.requires_grad, model.parameters()), lr=1e-3)
steps = 20; scheduler = torch.optim.lr_scheduler.CosineAnnealingLR(optimizer, steps, eta_min=1e-4)
exp(model, device, train_loader, test_loader, optimizer, scheduler, epochs=steps)
```

```
Epoch 0: train: loss 2.29156 acc 27.32% test: loss 13.8853 acc 23.59%
Epoch 1: train: loss 2.11928 acc 32.72% test: loss 2.04082 acc 25.71%
Epoch 2: train: loss 1.96341 acc 34.83% test: loss 1.65199 acc 41.84%
Epoch 3: train: loss 1.51059 acc 46.19% test: loss 1.29571 acc 53.45%
Epoch 4: train: loss 1.23441 acc 56.24% test: loss 1.04609 acc 63.09%
Epoch 5: train: loss 1.0076 acc 65.15% test: loss 0.948037 acc 67.18%
Epoch 6: train: loss 0.888561 acc 69.18% test: loss 0.796689 acc 72.47%
Epoch 7: train: loss 0.778699 acc 73.26% test: loss 0.754892 acc 74.10%
Epoch 8: train: loss 0.68487 acc 76.59% test: loss 0.697933 acc 76.37%
Epoch 9: train: loss 0.619202 acc 78.78% test: loss 0.669965 acc 77.55%
Epoch 10: train: loss 0.550703 acc 81.25% test: loss 0.644493 acc 78.92%
Epoch 11: train: loss 0.512075 acc 82.62% test: loss 0.624747 acc 79.98%
Epoch 12: train: loss 0.451228 acc 84.61% test: loss 0.717274 acc 76.57%
Epoch 13: train: loss 0.437956 acc 85.08% test: loss 0.690779 acc 78.11%
Epoch 14: train: loss 0.387881 acc 86.80% test: loss 0.642618 acc 80.10%
Epoch 15: train: loss 0.361026 acc 87.69% test: loss 0.618955 acc 80.55%
Epoch 16: train: loss 0.335962 acc 88.43% test: loss 0.623705 acc 81.00%
Epoch 17: train: loss 0.316203 acc 89.20% test: loss 0.641408 acc 81.49%
Epoch 18: train: loss 0.266387 acc 90.77% test: loss 0.677861 acc 80.38%
Epoch 19: train: loss 0.277275 acc 90.56% test: loss 0.605966 acc 82.17%
```

4 Ejercicio

Ejercicio: prueba aumento de datos con MNIST y FashionMNIST; considera, por ejemplo, las transformaciones siguientes

Rotación aleatoria: RandomRotation(degrees, interpolation = InterpolationMode.NEAREST, expand = False, center = None, fill = 0)

- `degrees` puede ser `int` ([-degrees, +degrees]) o secuencia ([min, max])
- `center` es el centro de rotación; `None` denota centro de la imagen

Afinidad aleatoria: RandomAffine(degrees, translate = None, scale = None, shear = None, interpolation = InterpolationMode.NEAREST, fill = 0, center = None)

- `degrees` puede ser `int` ([-degrees, +degrees]) o secuencia ([min, max])
- `translate` es una tupla con fracciones absolutas máximas para translaciones horizontales y verticales
- `scale` es una tupla que indica el intervalo del factor de escalado
- `shear` puede ser `int` ([-shear, +shear]) o secuencia ([min, max])

Recorte redimensionado aleatorio: RandomResizedCrop(size, scale=(0.08, 1.0), ratio=(0.75, 1.3333), interpolation=InterpolationMode.BILINEAR, antialias = True)

- `size` puede ser `int` (recorte cuadrado) o secuencia ([height, width])
- `ratio` es una tupla con cotas inferior y superior para la relación de aspecto aleatoria del recorte

In [ ]:
```
import datasets; from torch.utils.data import DataLoader
ds = datasets.load_dataset("ylecun/mnist").rename_columns({'image': 'X', 'label': 'y'})
train_ds = ds['train'].to_iterable_dataset(num_shards=1024).shuffle()
test_ds = ds['test'].to_iterable_dataset(num_shards=1024)

import numpy as np; import matplotlib.pyplot as plt; import torch; import timm; from exp import exp
device = torch.accelerator.current_accelerator().type if torch.accelerator.is_available() else "cpu"; device
model_name = "resnet50.fb_swsl_ig1b_ft_in1k" # avg_top1 71.5 param_count 25.56
model = timm.create_model(model_name, pretrained=True, num_classes=10, in_chans=1).to(device)

from torchvision.transforms import v2
train_transform = v2.Compose([v2.ToImage(), v2.RandomRotation(10),
    v2.RandomAffine(degrees=0, translate=(.1, .1)), v2.ToDtype(torch.float32, scale=True)])
def train_prep(example):
    return { 'X' : train_transform(example['X']), 'y' : example['y']}
train_ds = train_ds.map(train_prep, batched=True)
train_loader = DataLoader(train_ds, batch_size=32)
test_transform = v2.Compose([v2.ToImage(), v2.ToDtype(torch.float32, scale=True)])
def test_prep(example):
    return { 'X' : test_transform(example['X']), 'y' : example['y']}
test_ds = test_ds.map(test_prep, batched=True)
test_loader = DataLoader(test_ds, batch_size=32)
optimizer = torch.optim.AdamW(filter(lambda p: p.requires_grad, model.parameters()), lr=1e-3)
steps = 15; scheduler = torch.optim.lr_scheduler.CosineAnnealingLR(optimizer, steps, eta_min=1e-4)
exp(model, device, train_loader, test_loader, optimizer, scheduler, epochs=steps)
```

```
Epoch 0: train: loss 0.666053 acc 86.59% test: loss 0.140298 acc 96.27%
Epoch 1: train: loss 0.302255 acc 93.16% test: loss 0.154909 acc 95.18%
Epoch 2: train: loss 0.181704 acc 95.32% test: loss 0.0554843 acc 98.57%
Epoch 3: train: loss 0.104016 acc 97.08% test: loss 0.0453219 acc 98.50%
Epoch 4: train: loss 0.1431 acc 96.54% test: loss 0.0389119 acc 98.87%
Epoch 5: train: loss 0.10172 acc 97.16% test: loss 0.0511085 acc 98.46%
Epoch 6: train: loss 0.0915378 acc 97.48% test: loss 0.126014 acc 96.49%
Epoch 7: train: loss 0.0949098 acc 97.35% test: loss 0.0319092 acc 99.12%
Epoch 8: train: loss 0.0700925 acc 98.04% test: loss 0.0316818 acc 98.98%
Epoch 9: train: loss 0.0766725 acc 97.93% test: loss 0.0274909 acc 99.11%
Epoch 10: train: loss 0.0626611 acc 98.20% test: loss 0.0250941 acc 99.30%
Epoch 11: train: loss 0.0569222 acc 98.42% test: loss 0.0346619 acc 98.95%
Epoch 12: train: loss 0.0483397 acc 98.57% test: loss 0.0430244 acc 98.82%
Epoch 13: train: loss 0.0505372 acc 98.60% test: loss 0.0252422 acc 99.31%
Epoch 14: train: loss 0.0480878 acc 98.61% test: loss 0.0296231 acc 99.08%
```

In [ ]:
```
import datasets; from torch.utils.data import DataLoader
ds = datasets.load_dataset("zalando-datasets/fashion_mnist").rename_columns({'image': 'X', 'label': 'y'})
train_ds = ds['train'].to_iterable_dataset(num_shards=1024).shuffle()
test_ds = ds['test'].to_iterable_dataset(num_shards=1024)

import numpy as np; import matplotlib.pyplot as plt; import torch; import timm; from exp import exp
device = torch.accelerator.current_accelerator().type if torch.accelerator.is_available() else "cpu"; device
model_name = "resnet50.fb_swsl_ig1b_ft_in1k" # avg_top1 71.5 param_count 25.56
model = timm.create_model(model_name, pretrained=True, num_classes=10, in_chans=1).to(device)

from torchvision.transforms import v2
train_transform = v2.Compose([v2.ToImage(), v2.RandomRotation(degrees=3),
    v2.RandomAffine(degrees=0, translate=(.03, .03)), v2.ToDtype(torch.float32, scale=True)])
def train_prep(example):
    return { 'X' : train_transform(example['X']), 'y' : example['y']}
train_ds = train_ds.map(train_prep, batched=True)
train_loader = DataLoader(train_ds, batch_size=32)
test_transform = v2.Compose([v2.ToImage(), v2.ToDtype(torch.float32, scale=True)])
def test_prep(example):
    return { 'X' : test_transform(example['X']), 'y' : example['y']}
test_ds = test_ds.map(test_prep, batched=True)
test_loader = DataLoader(test_ds, batch_size=32)
optimizer = torch.optim.AdamW(filter(lambda p: p.requires_grad, model.parameters()), lr=1e-3)
steps = 15; scheduler = torch.optim.lr_scheduler.CosineAnnealingLR(optimizer, steps, eta_min=1e-4)
exp(model, device, train_loader, test_loader, optimizer, scheduler, epochs=steps)
```

```
Epoch 0: train: loss 0.745859 acc 77.98% test: loss 0.538284 acc 80.31%
Epoch 1: train: loss 0.436634 acc 85.30% test: loss 0.331036 acc 88.01%
Epoch 2: train: loss 0.465623 acc 85.50% test: loss 0.334169 acc 88.10%
Epoch 3: train: loss 0.322249 acc 88.43% test: loss 0.32987 acc 87.55%
Epoch 4: train: loss 0.301147 acc 89.18% test: loss 0.269646 acc 89.96%
Epoch 5: train: loss 0.289478 acc 89.55% test: loss 0.27194 acc 90.24%
Epoch 6: train: loss 0.261156 acc 90.55% test: loss 0.238028 acc 91.23%
Epoch 7: train: loss 0.267273 acc 90.48% test: loss 0.246614 acc 91.15%
Epoch 8: train: loss 0.230157 acc 91.60% test: loss 0.235612 acc 91.53%
Epoch 9: train: loss 0.22451 acc 91.85% test: loss 1.38171 acc 89.05%
Epoch 10: train: loss 0.207452 acc 92.39% test: loss 0.267043 acc 90.40%
Epoch 11: train: loss 0.214625 acc 92.27% test: loss 0.232371 acc 91.56%
Epoch 12: train: loss 0.191395 acc 92.95% test: loss 0.254697 acc 91.41%
Epoch 13: train: loss 0.184292 acc 93.19% test: loss 0.232137 acc 91.85%
Epoch 14: train: loss 0.174366 acc 93.55% test: loss 0.274674 acc 90.84%
```